COMPUTER ENGINEERING
QUEZON CITY

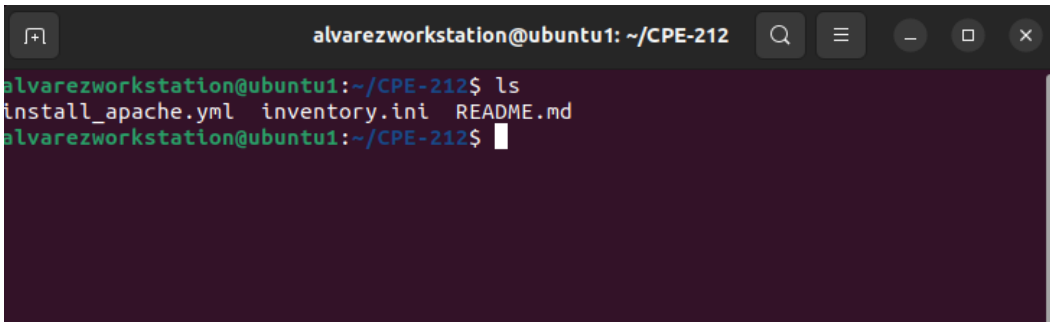
COURSE CODE/TITLE:	CPE212 - Automating Server Management	**SECTION:**	CPE31S2
DATE PERFORMED:	27/08/2026	**DATE SUBMITTED:**	27/08/2026
NAME:	Alvarez, Erick Jefferson A	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 5.1 **Implement Ansible roles in playbooks**

1. PROCEDURE OUTPUT

Task 1: Use when command for different distributions

1. In the local machine, make sure you are in the local repository directory (*CPE232_yourname*). Issue the command git pull. When prompted, enter the correct passphrase or password. Describe what happened when you issue this command. Did something happen? Why?



- I actually created this on my laptop then when I typed ls, I saw that my repository already existed on this machine so when I git pull it inside the repository, all files are pulled into this machine.

2. Edit the inventory file and add the IP address of the Centos VM. Issue the command we used to execute the playbook (the one we used in the last activity): *ansible-playbook --ask-become-pass install_apache.yml*. After executing this command, you may notice that it did not become successful in the Centos VM. You can see that the Centos VM has failed=1. Only the two remote servers have been changed. The reason is that Centos VM does not support "apt" as the package manager. The default package manager for Centos is "yum."

```
TASK [Install Apache2] *****************************************************
[WARNING]: Updating cache and auto-installing missing dependency: python3-a
fatal: [192.168.56.122]: FAILED! => {"changed": false, "cmd": "apt-get upda
"msg": "[Errno 2] No such file or directory: b'apt-get'", "rc": 2}
ok: [192.168.56.121]
ok: [192.168.56.120]

TASK [Install PHP] *********************************************************
changed: [192.168.56.121]
changed: [192.168.56.120]

PLAY RECAP *****************************************************************
192.168.56.120             : ok=3    changed=1    unreachable=0    failed=0
kipped=0    rescued=0    ignored=0
192.168.56.121             : ok=3    changed=1    unreachable=0    failed=0
kipped=0    rescued=0    ignored=0
192.168.56.122             : ok=1    changed=0    unreachable=0    failed=1
kipped=0    rescued=0    ignored=0

alvarezworkstation@ubuntu1:~/CPE-212$
```

3. Edit the *install_apache.yml* file and insert the lines shown below.

```
---
- hosts: all
  become: true
  tasks:

  - name: update repository index
    apt:
      update_cache: yes
    when: ansible_distribution == "Ubuntu"

  - name: install apache2 package
    apt:
      name: apache2
    when: ansible_distribution == "Ubuntu"

  - name: add PHP support for apache
    apt:
      name: libapache2-mod-php
    when: ansible_distribution == "Ubuntu"
```

Make sure to save the file and exit.

Run *ansible-playbook --ask-become-pass install_apache.yml* and describe the result.

```
TASK [install apache2 package] *****************************************
ok: [192.168.56.121]
ok: [192.168.56.120]

TASK [add PHP support for apache] **************************************
changed: [192.168.56.121]
changed: [192.168.56.120]

PLAY RECAP *************************************************************
192.168.56.120             : ok=7    changed=2    unreachable=0    failed=0
kipped=0    rescued=0    ignored=0
192.168.56.121             : ok=7    changed=2    unreachable=0    failed=0
kipped=0    rescued=0    ignored=0
192.168.56.122             : ok=1    changed=0    unreachable=0    failed=1
kipped=0    rescued=0    ignored=0

alvarezworkstation@ubuntu1:~/CPE-212$
```

- Same result as number two, centOS is still not working because of apt

If you have a mix of Debian and Ubuntu servers, you can change the configuration of your playbook like this.

- name: update repository index
  apt:
    update_cache: yes
  when: ansible_distribution in [“Debian”, “Ubuntu]

Note: This will work also if you try. Notice the changes are highlighted.

4. Edit the *install_apache.yml* file and insert the lines shown below.

```
---
- hosts: all
  become: true
  tasks:

  - name: update repository index
    apt:
      update_cache: yes
    when: ansible_distribution == "Ubuntu"

  - name: install apache2 package
    apt:
      name: apache2
      stae: latest
    when: ansible_distribution == "Ubuntu"

  - name: add PHP support for apache
    apt:
      name: libapache2-mod-php
      state: latest
    when: ansible_distribution == "Ubuntu"

  - name: update repository index
    dnf:
      update_cache: yes
    when: ansible_distribution == "CentOS"

  - name: install apache2 package
    dnf:
      name: httpd
      state: latest
    when: ansible_distribution == "CentOS"

  - name: add PHP support for apache
    dnf:
      name: php
      state: latest
    when: ansible_distribution == "CentOS"
```

Make sure to save and exit.

```
PLAY RECAP *********************************************************************
192.168.56.105             : ok=4    changed=1    unreachable=0    failed=0
kipped=3    rescued=0    ignored=0
192.168.56.107             : ok=4    changed=1    unreachable=0    failed=0
kipped=3    rescued=0    ignored=0
192.168.56.108             : ok=4    changed=2    unreachable=0    failed=0
kipped=3    rescued=0    ignored=0

erickalvarez@alvarezworkstation:~/CPE-212$
```

Run *ansible-playbook --ask-become-pass install_apache.yml* and describe the result.

- It automatically downloads php and apache for CentOS because we used dnf instead of apt

5. To verify the installations, go to CentOS VM and type its IP address on the browser. Was it successful? The answer is no. It's because the httpd service or the Apache HTTP server in the CentOS is not yet active. Thus, you need to activate it first.

   5.1 To activate, go to the CentOS VM terminal and enter the following:
   systemctl status httpd
   The result of this command tells you that the service is inactive.

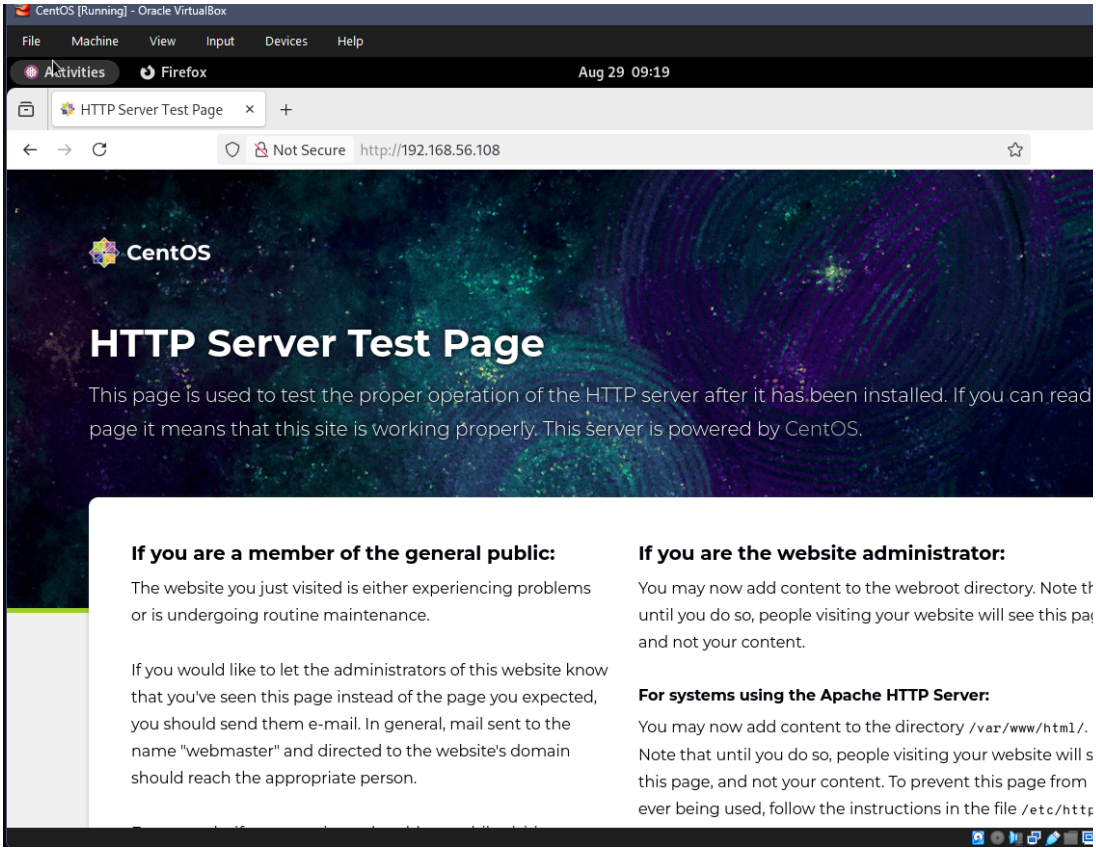
   5.2 Issue the following command to start the service:
   sudo systemctl start httpd
   (When prompted, enter the sudo password)
   sudo firewall-cmd --add-port=80/tcp
   (The result should be a success)

```
erickalvarez@alvarezworkstation:~/CPE-212$ ssh ealvarez@192.168.56.10
Activate the web console with: systemctl enable --now cockpit.socket

Last login: Sat Aug 29 09:13:51 2026 from 192.168.56.106
[ealvarez@localhost ~]$ systemctl status httpd
○ httpd.service - The Apache HTTP Server
     Loaded: loaded (/usr/lib/systemd/system/httpd.service; disabled;
    Drop-In: /usr/lib/systemd/system/httpd.service.d
             └─php-fpm.conf
     Active: inactive (dead)
       Docs: man:httpd.service(8)

[ealvarez@localhost ~]$ sudo firewall-cmd --add-port=80/tcp
[sudo] password for ealvarez:
success
[ealvarez@localhost ~]$
```

5.3 To verify the service is already running, go to CentOS VM and type its IP address on the browser. Was it successful? (Screenshot the browser)



- It works after doing “systemctl start httpd”

Task 2: Refactoring playbook

This time, we want to make sure that our playbook is efficient and that the codes are easier to read. This will also makes run ansible more quickly if it has to execute fewer tasks to do the same thing.

1. Edit the playbook *install_apache.yml*. Currently, we have three tasks targeting our Ubuntu machines and 3 tasks targeting our CentOS machine. Right now, we try to consolidate some tasks that are typically the same. For example, we can consolidate two plays that install packages. We can do that by creating a list of installation packages as shown below:

```
---
- hosts: all
  become: true
  tasks:

  - name: update repository index Ubuntu
    apt:
      update_cache: yes
    when: ansible_distribution == "Ubuntu"

  - name: install apache2 and php packages for Ubuntu
    apt:
      name:
        - apache2
        - libapache2-mod-php
      state: latest
    when: ansible_distribution == "Ubuntu"

  - name: update repository index for CentOS
    dnf:
      update_cache: yes
    when: ansible_distribution == "CentOS"

  - name: install apache and php packages for CentOS
    dnf:
      name:
        - httpd
        - php
      state: latest
    when: ansible_distribution == "CentOS"
```

Make sure to save the file and exit.

Run *ansible-playbook --ask-become-pass install_apache.yml* and describe the result.

```
PLAY RECAP *********************************************************************
192.168.56.105             : ok=3    changed=1    unreachable=0    failed=
kipped=2    rescued=0    ignored=0
192.168.56.107             : ok=3    changed=1    unreachable=0    failed=
kipped=2    rescued=0    ignored=0
192.168.56.108             : ok=3    changed=0    unreachable=0    failed=
kipped=2    rescued=0    ignored=0

erickalvarez@alvarezworkstation:~/CPE-212$
```

- Same result as before, failed=0 on all hosts just fewer tasks now that apache and php install together per OS.

2. Edit the playbook *install_apache.yml* again. In task 2.1, we consolidated the plays into one play. This time we can actually consolidated everything in just 2 plays. This can be done by removing the update repository play and putting the command *update_cache: yes* below the command *state: latest*. See below for reference:

```
---
- hosts: all
  become: true
  tasks:

   - name: install apache2 and php packages for Ubuntu
     apt:
      name:
        - apache2
        - libapache2-mod-php
      state: latest
      update_cache: yes
     when: ansible_distribution == "Ubuntu"

   - name: install apache and php packages for CentOS
     dnf:
       name:
         - httpd
         - php
       state: latest
       update_cache: yes
     when: ansible_distribution == "CentOS"
```

Make sure to save the file and exit.

Run *ansible-playbook --ask-become-pass install_apache.yml* and describe the result.

```
PLAY RECAP *********************************************************************
192.168.56.105             : ok=2    changed=0    unreachable=0    failed=
kipped=1    rescued=0    ignored=0
192.168.56.107             : ok=2    changed=0    unreachable=0    failed=
kipped=1    rescued=0    ignored=0
192.168.56.108             : ok=2    changed=0    unreachable=0    failed=
kipped=1    rescued=0    ignored=0

erickalvarez@alvarezworkstation:~/CPE-212$
```

- Still failed=0. Playbook is shorter since the cache update is now part of the install task.

3. Finally, we can consolidate these 2 plays in just 1 play. This can be done by declaring variables that will represent the packages that we want to install. Basically, the apache_package and php_package are variables. The names are arbitrary, which means we can choose different names. We also take out the line when: ansible_distribution. Edit the playbook *install_apache.yml* again and make sure to follow the below image. Make sure to save the file and exit.

```
---
- hosts: all
  become: true
  tasks:

  - name: install apache and php
    apt:
      name:
        - "{{ apache_package }}"
        - "{{ php_package }}"
      state: latest
      update_cache: yes
```

Run *ansible-playbook --ask-become-pass install_apache.yml* and describe the result.

```
PLAY RECAP *********************************************************************
192.168.56.105             : ok=1    changed=0    unreachable=0    failed=1
kipped=0    rescued=0    ignored=0
192.168.56.107             : ok=1    changed=0    unreachable=0    failed=1
kipped=0    rescued=0    ignored=0
192.168.56.108             : ok=1    changed=0    unreachable=0    failed=1
```

- Failed on all hosts. The variables apache_package and php_package weren't defined anywhere yet, so Ansible didn't know what to install.

4. Unfortunately, task 2.3 was not successful. It's because we need to change something in the inventory file so that the variables we declared will be in place. Edit the *inventory* file and follow the below configuration:

```
192.168.56.120 apache_package=apache2 php_package=libapache2-mod-php
192.168.56.121 apache_package=apache2 php_package=libapache2-mod-php
192.168.56.122 apache_package=httpd php_package=php
```

Make sure to save the *inventory* file and exit.

Finally, we still have one more thing to change in our *install_apache.yml* file. In task 2.3, you may notice that the package is assign as apt, which will not run in CentOS. Replace the *apt* with *package*. Package is a module in ansible that is generic, which is going to use whatever package manager the underlying host or the target server uses. For Ubuntu it will automatically use *apt*, and for CentOS it will automatically use *dnf*. Make sure to save the file and exit. For more details about the ansible package, you may refer to this documentation: ansible.builtin.package – Generic OS package manager — Ansible Documentation

Run *ansible-playbook --ask-become-pass install_apache.yml* and describe the result.

```
PLAY RECAP *********************************************************************
192.168.56.105             : ok=2    changed=0    unreachable=0    failed=0    s
kipped=0    rescued=0    ignored=0
192.168.56.107             : ok=2    changed=0    unreachable=0    failed=0    s
kipped=0    rescued=0    ignored=0
192.168.56.108             : ok=2    changed=0    unreachable=0    failed=0    s
kipped=0    rescued=0    ignored=0

erickalvarez@alvarezworkstation:~/CPE-212$
```

- After adding the variables to the inventory and switching apt to package, it ran successfully with failed=0 on all three hosts now works for both Ubuntu and CentOS.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Supplementary Activity:

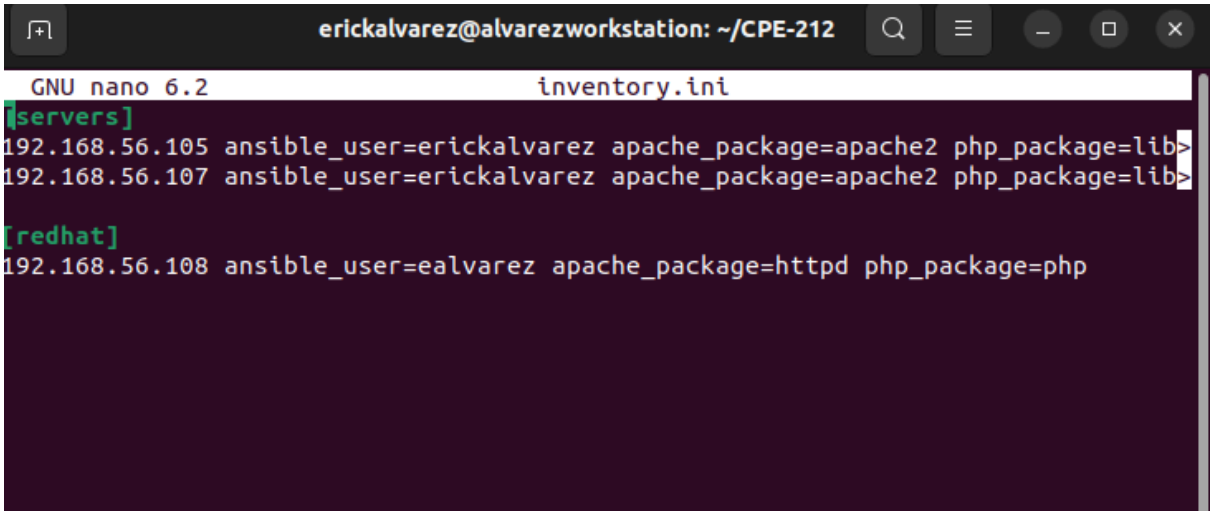
1. Create a playbook that could do the previous tasks in Red Hat OS.

Output:

```
PLAY RECAP *********************************************************************
192.168.56.108             : ok=2    changed=0    unreachable=0    failed=0    s
kipped=0    rescued=0    ignored=0

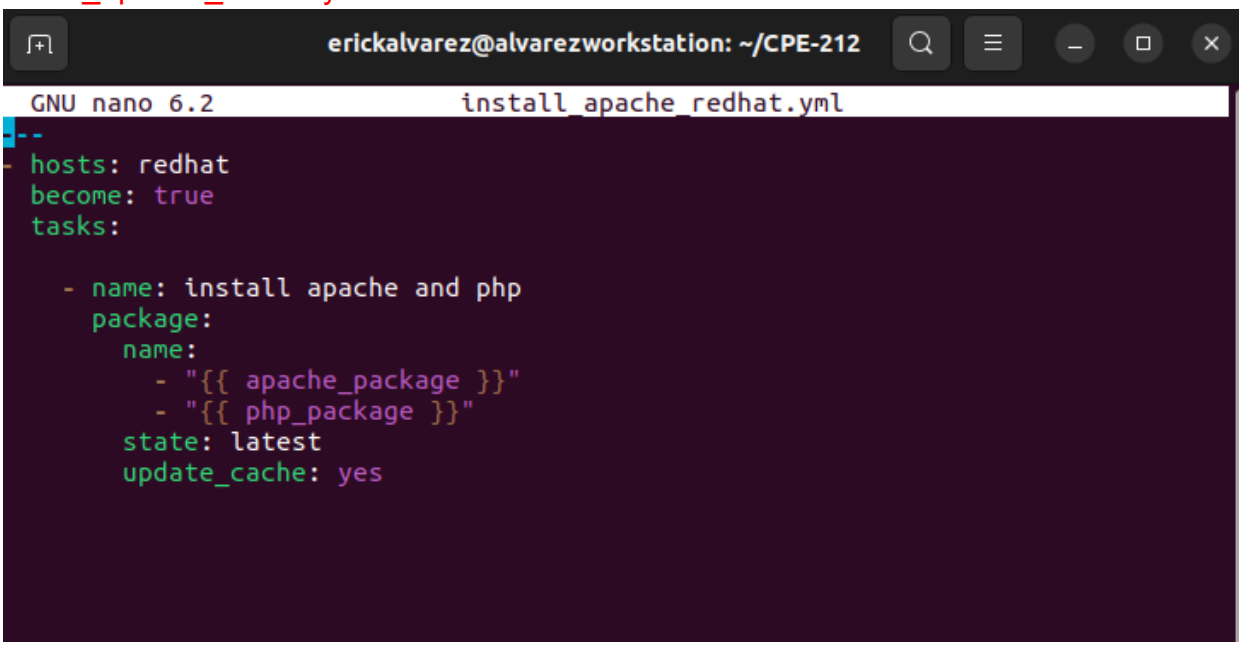
erickalvarez@alvarezworkstation:~/CPE-212$ s
```

Inventory:



- I separated my CentOS since CentOS is also a Red Hat OS, that's why I used it as my OS for Red Hat.

Install_apache_redhat.yml



- For this, I created a separate playbook for Red Hat. As you can see, my host only points to redhat, so that it only reads the redhat group in my inventory.

Reflections:

Answer the following:

1. Why do you think refactoring of playbook codes is important?
   - Refactoring keeps the playbook shorter, easier to read, and faster to run since it avoids repeating the same tasks for each OS.

2. When do we use the “when” command in playbook?

   - We use when to make a task run only on certain hosts, based on a condition like the OS type so it doesn't fail on machines where that task doesn't apply.

Conclusion:
This activity taught me how to simplify a messy playbook using variables instead of repeating tasks per OS. I also learned that small mistakes, like a missing variable or a typo, can break the whole run, so accuracy matters as much as writing clean code.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Claude AI**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Error explanation**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **AI was used to only explain errors on playbooks.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.’s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.